

AI-Native Workflow

Hierarchical Intent, Human-Attention-Aware Oversight, and Evidence-Gated Recovery for
Long-Horizon Language-Agent Execution

Yuhe Sui

Version 5.6.37 — 3 September 2026

Abstract

Researchers already compose frontier AI surfaces informally: persistent chat for planning and interpretation, research systems for external evidence, and coding agents for repository execution. The operational bottleneck is increasingly the human attention required to keep those surfaces coherent across long-horizon work. This report specifies AI-Native Workflow, a package-resident control layer whose canonical hierarchy is *Goal* $\rightarrow$ *Plan* $\rightarrow$ *Phase* $\rightarrow$ *durable intent contract (DIC)* $\rightarrow$ *disposable execution prompt stack (EPS)* $\rightarrow$ *Prompt/Run* $\rightarrow$ *Action*. Main is a persistent planning and decision chat; Theory and Experiment are optional specialist reasoning workstreams; a repository-aware coding-agent Orchestrator realizes each approved Phase through model-tuned EPS attempts and bounded subagents. The DIC is both a semantic contract and a human-intelligibility interface: it preserves one Primary Aim, prioritized requests, the durable dependency/parallelism graph, evidence requirements, and material escalation boundaries while hiding routine execution noise. We formalize semantic refinement, contract composition, dependency-local invalidation, imperfect verification, context projection, human-checkpoint complexity, and a human-attention intensity objective: supervisory attention divided by verified research progress. Under complete read/write declarations and versioned evidence, a repair need invalidate only its dynamic dependency cone, and that cone is worst-case tight. Conversely, evidence aliasing prevents universal verifier correctness. The attention objective and long-horizon performance benefits are evaluation targets, not established empirical gains; we define a matched-budget programme that measures both task quality and human-attention burden while varying effective horizon and injected faults.

Contents

1	Introduction	3
2	The Long-Horizon Agent Problem	4
2.1	Execution traces and drift	4
2.2	Characteristic failures	4
3	Hierarchical Execution Model	4
3.1	Levels and roles	4
3.2	Plans and phases	5
3.3	Durable intent contracts	5
3.4	EPS, prompt/runs, and actions	5
4	Intent and Contract Refinement	6
4.1	Trace semantics	6
4.2	Acceptance predicates and acceptable-state sets	6
4.3	Compositional satisfaction	6

5	Verification and Replanning	7
5.1	Closed-loop operator	7
5.2	Imperfect verification	7
5.3	Repair versus replan	7
6	Dependency Locality and Failure Propagation	8
6.1	Declared footprints	8
6.2	Propagation bounds	8
7	Context and Persistent State	9
7.1	Package as authority	9
7.2	Context projection	9
7.3	DIC as a human-attention projection	9
8	Human Oversight Across Timescales	9
8.1	Human-attention intensity	9
8.2	Governance complexity	10
8.3	Checkpoint interval under imperfect verification	10
9	Main, Theory, Experiment, and Coding-Agent Orchestration	11
10	Reference Implementation	11
10.1	Authority and files	11
10.2	State transitions	11
10.3	Compatibility	12
10.4	Conformance coverage	12
11	Formal Properties and Limits	12
11.1	What is guaranteed by construction	12
11.2	Observability limit	12
11.3	Open-world and adversarial limits	13
11.4	Novelty limit	13
12	Empirical Hypotheses and Evaluation Design	13
12.1	Central hypotheses	13
12.2	Conditions	13
12.3	Task factors and faults	13
12.4	Outcomes and analysis	13
12.5	Current evidence boundary	14
13	Related Work	14
14	Limitations	14
15	Conclusion	15
A	Claim classification summary	15
B	Required experiment record	15
C	Ordinary execution brief	16

1 Introduction

Researchers increasingly use several AI systems within one project: a persistent chat may plan or interpret, a research surface may gather literature, and a coding agent may modify the repository and run experiments. The difficult regime is therefore no longer a single model call. It is a programme extending across dependent decisions, artifacts, model contexts, execution surfaces, and human review cycles. The researcher often remains the hidden coordinator: remembering which result is current, moving context between tools, checking whether an assumption changed, reconciling branches, and reconstructing what actually ran.

The practical objective is not simply to minimize wall-clock time. Additional verification or parallel execution can increase total inference and sometimes latency. The stronger target is to reduce *human attention required per unit of trustworthy research progress*: routine coordination and state reconstruction should move into durable machine-managed surfaces, while human attention remains concentrated on scientific objectives, surprising evidence, material methodological changes, claim boundaries, and external responsibility.

Existing agents already plan, reflect, search, use memory, and communicate in manager-worker arrangements (Yao et al., 2023b,a; Besta et al., 2024; Shinn et al., 2023; Wu et al., 2023; Hong et al., 2024). The claim is therefore deliberately narrower than “hierarchical planning is new.” AI-Native Workflow formalizes an informal multi-surface research workflow into an explicit *authority, refinement, and evidence protocol*: persistent intent is approved at a human timescale, refined through bounded machine contracts, linked to evidence, and repaired at the smallest dependency-valid scope. Hierarchical planning becomes operational inside current chat, research, and repository-aware coding-agent settings rather than remaining implicit in one long conversation.

AI-Native Workflow 5.6.37 implements that protocol inside the working package. Only `.ai-workflow/` is the operative runtime. Main is the persistent user-facing planning and decision chat. Theory and Experiment are optional specialist reasoning workstreams. Research is an advisory evidence pass returning to Main for reconciliation. A repository-aware coding-agent Orchestrator (for example Codex or Claude Code) consumes the approved whole-Phase DIC, realizes it through one or more disposable EPS attempts, routes bounded Workers/subagents, integrates evidence, performs local repair, and dispatches verification. Human governance is separated from machine execution: ordinary coordination stays below the human-attention boundary unless scientific meaning, authority, or decisive evidence changes.

The report makes five contributions. First, it gives an executable specification of the hierarchy and state machine. Second, it treats the DIC as both a semantic contract and a compact human-attention interface that preserves a Phase Primary Aim and priority structure. Third, it formalizes what intent preservation and compositional satisfaction require rather than treating hierarchy as self-justifying. Fourth, it proves dependency-local recovery and tightness results, an imperfect-verification bound, a checkpoint-cost proposition, and an observability impossibility. Fifth, it defines an evaluation programme that tests both long-horizon task quality and human-attention intensity against matched-budget alternatives. The formal results adapt established contract, incremental-computation, reliability, and partial-observability ideas; the intended contribution is their integrated operationalization and empirical test in language-agent harnesses.

Claim discipline. Every substantial statement is assigned one of: definition; invariant by construction; theorem or proposition under explicit assumptions; empirical hypothesis; conjecture; or unsupported and removed. The machine-readable ledger in `research/CLAIM_LEDGER.json` is authoritative for manuscript integration.

2 The Long-Horizon Agent Problem

2.1 Execution traces and drift

Let $\mathcal{S}$ be the joint repository–environment state, including files, tool state, external observations that have been durably recorded, and governance metadata. A primitive action a induces a possibly stochastic transition kernel

$$K_a : \mathcal{S} \rightsquigarrow \mathcal{S} \times \mathcal{E},$$

where $\mathcal{E}$ contains logs, test outputs, citations, measurements, or human decisions. A run is a trace $\tau = (s_0, a_1, e_1, s_1, \dots, a_T, e_T, s_T) \in \mathcal{T}$. A global intent is not merely a natural-language string: it comprises a terminal goal predicate $G(s_T)$, trace constraints $J(\tau)$, authority boundaries, and evidence requirements.

Long horizon is multidimensional. Sequence length matters, but so do dependency depth, branching, delayed reuse of earlier artifacts, interacting constraints, state hidden outside the current context, and the distance between a fault and the observation that reveals it. We call the resulting quantity *effective task horizon*. It is an empirical construct to be operationalized by a task dependency graph rather than equated with prompt length or token count.

2.2 Characteristic failures

The architecture targets seven recurrent failure classes.

1. **Intent drift**: a local objective remains plausible but no longer refines the approved goal.
2. **Constraint–objective inversion (defensive drift)**: a safeguard such as “avoid unsupported claims” gradually becomes the effective objective, producing maximally defensible work rather than the strongest correct result.
3. **Salience flattening**: many locally reasonable branches receive similar weight until the project loses a clear primary contribution or central question.
4. **Authority drift**: a machine makes a scientific, release, or external decision that required human authority.
5. **Unsupported completion**: an artifact is marked complete without evidence satisfying its acceptance predicate.
6. **Stale dependency propagation**: a repaired upstream object leaves downstream artifacts and evidence incorrectly trusted.
7. **Context fracture**: a fresh session lacks the assumptions, decisions, or evidence needed to continue safely.

The first three are semantic failures: every local response may be coherent while the programme’s center of gravity changes. The DIC therefore distinguishes one explicit Primary Aim from constraints, supporting questions, optional branches, non-goals, and deferred work. Constraints bound the search; they should not silently replace its objective.

3 Hierarchical Execution Model

3.1 Levels and roles

The canonical levels are shown in [Table 1](#).

Main owns overall authority, architecture, integration, state, and publication. Theory owns formal reasoning, literature and novelty analysis, propositions, and counterexamples. Experiment owns benchmark design, implementation, statistics, and empirical evidence. Search and research are capabilities available to Theory and Experiment. A workstream may delegate bounded actions, but it retains its DIC and integration responsibility.

Level	Lifetime	Semantic responsibility	Normal authority
Goal	programme	global outcome, constraints, authority	human + Main
Plan	several phases	strategic DAG and checkpoints	human approval, Main ownership
Phase	milestone	coherent research milestone	Main, with Theory/Experiment support
DIC	phase/repair-stable	Primary Aim, acceptance, evidence, durable execution graph	Main
EPS	disposable	JIT realization of ready DIC plan nodes	coding-agent Orchestrator
Prompt/Run	attempt-local	bounded model execution and context	Orchestrator/Worker/subagent
Action	primitive	tool use, edit, search, experiment, test	executor/tool

Table 1: The hierarchy separates durable semantics from disposable operations.

3.2 Plans and phases

Definition 3.1 (Plan). *A Plan is a tuple $P = (V, E, \prec, \Gamma, h)$ where (V, E) is a directed acyclic graph of phases, $\prec$ is a declared execution order compatible with E , Γ is the governance policy, and h is the periodic human-checkpoint interval. Each phase $v \in V$ has an owner, objective, dependency set, materiality class, DIC pointer, status, and evidence index.*

A Plan is the normal human approval object. A Phase is a machine milestone, not ordinarily a separate human approval request. The runtime permits automatic progression to a dependency-ready ordinary phase until a checkpoint becomes due or a material escalation opens.

3.3 Durable intent contracts

Definition 3.2 (DIC). *A durable intent contract for phase v is*

$$C_v = (D_v, I_v, A_v, W_v, \mathcal{U}_v, \mathcal{X}_v, \mathcal{R}_v),$$

where $D_v \subseteq \mathcal{S}$ is a precondition domain, $I_v \subseteq \mathcal{T}$ is a trace invariant, $A_v \subseteq \mathcal{S}$ is an acceptable post-state set, $W_v : \mathcal{S} \times \mathcal{E}^* \rightarrow \{0, 1\}$ is an evidence predicate, $\mathcal{U}_v$ is the authority/input set, $\mathcal{X}_v$ is the escalation set, and $\mathcal{R}_v$ is the declared dependency/read footprint.

The runtime materializes one DIC package per Phase rather than a sequence of stepwise DICs. The DIC is deliberately dual-use: a machine contract and a human-intelligibility projection. Required views are `README.md` (Primary Aim, priority/salience structure, authority, and compact orientation), `REQUESTS.md` (prioritized outcomes and acceptance requirements), and `PLAN.md` (the durable execution graph, including prompt identifiers, dependencies, parallel subagent batches, merge points, and conditional repair/review branches). Optional `ARCHITECTURE.md`, `TEST_MATRIX.md`, and user-defined flat Markdown views expose additional angles on the same contract. A machine `MANIFEST.json` indexes these views and content hashes. A DIC survives local repair. If the subgoal, acceptance meaning, frozen protocol, or authority changes materially, the correct operation is replan or escalate rather than silently mutating the contract.

3.4 EPS, prompt/runs, and actions

The DIC execution plan is durable, while an EPS is a disposable just-in-time instantiation of selected plan nodes against the current repository state, model/context, attempt, and repair history. An EPS may name prompt/run IDs, dependencies, parallel groups, subagent delegations,

expected outputs, local checks, and stop conditions, and it may materialize executable prompt files. Ordinary failure or retry therefore creates a new EPS/run instance without changing the DIC unless the Phase contract or durable execution design itself changes. An Action record links to an EPS and, when applicable, a prompt/run node, while preserving DIC, Phase, Plan, result, and evidence lineage.

4 Intent and Contract Refinement

4.1 Trace semantics

Associate each object X in the hierarchy with an admitted trace set $\llbracket X \rrbracket \subseteq \mathcal{T}_X$. Adjacent levels may use different state descriptions, so let $\rho_{XY} : \mathcal{T}_X \rightarrow \mathcal{T}_Y$ be a declared abstraction map.

Definition 4.1 (Semantic refinement). X *refines* Y , written $X \preceq_\rho Y$, if

$$\rho_{XY}(\llbracket X \rrbracket) \subseteq \llbracket Y \rrbracket.$$

This definition makes intent preservation a semantic inclusion claim, not a similarity judgment between prompt strings. In a practical package, refinement is supported by contract predicates, evidence, tests, and review; it is rarely decided by a complete semantic oracle.

Proposition 4.2 (Refinement-chain preservation). *If $X_0 \preceq X_1 \preceq \dots \preceq X_m$ and the abstraction maps compose on admitted traces, then $X_0 \preceq X_m$.*

Proof. Repeated image inclusion gives $\rho_{0m}(\llbracket X_0 \rrbracket) \subseteq \rho_{1m}(\llbracket X_1 \rrbracket) \subseteq \llbracket X_m \rrbracket$. $\square$

The proposition is a consequence of the definition. It does not show that a model-generated child actually refines its parent; that requires a sound certificate or verifier.

4.2 Acceptance predicates and acceptable-state sets

For a completed phase trace τ_v ending in state s' , define

$$\text{Acc}_{C_v}(\tau_v, e_v) = \mathbf{1}\{s' \in A_v, \tau_v \in I_v, W_v(s', e_v) = 1\}.$$

Acceptance separates the desired state from evidence that warrants believing the state was reached. A unit test may be strong evidence for a deterministic software property and weak evidence for a broad scientific claim. The DIC must state the connection.

4.3 Compositional satisfaction

For each phase v , let F_v be its implemented transition relation. Let $\text{Pred}(v)$ be its predecessors. Write Pre_v and Post_v for its pre/post predicates and Frame_v for keys it promises not to alter.

Assumption 4.3 (Contract soundness). *For every phase v , every admitted execution satisfies the Hoare-style relation $\{\text{Pre}_v \wedge I\} F_v \{\text{Post}_v \wedge I\}$, and its evidence predicate is sound for the asserted postcondition.*

Assumption 4.4 (Edge compatibility). *For each non-source v , the conjunction of relevant predecessor postconditions implies Pre_v . Concurrent or unordered branches either have disjoint writes or satisfy an explicit commutativity/noninterference condition.*

Theorem 4.5 (Compositional satisfaction). *For an acyclic Plan satisfying contract soundness and edge compatibility, if the initial state satisfies all source preconditions and the conjunction of terminal postconditions implies the global goal G , then every admitted complete topological execution terminates in a state satisfying G and the global invariant I .*

Proof. Proceed by induction over a topological order. Source preconditions hold initially. At step v , all predecessors have executed; edge compatibility and noninterference imply Pre_v , while the induction hypothesis gives I . Contract soundness yields Post_v and preserves I . After all nodes, all terminal postconditions and I hold; the terminal implication gives G . $\square$

The assumptions are substantial. Hidden dependencies, unsound evidence, stochastic tools outside the contract, or incompatible branches invalidate the conclusion. The theorem is a precise restatement of contract composition in this setting, not a claim that LLM agents automatically satisfy it (Hoare, 1969).

5 Verification and Replanning

5.1 Closed-loop operator

Verification consumes the DIC, current state, emitted evidence, and relevant dependency versions. It returns

$$\mathcal{V}(C_v, s, e) \in \{\text{accept, repair, replan, escalate}\}.$$

Accept closes the DIC. **Repair** retains the DIC and regenerates EPS. **Replan** changes the decomposition or contract and therefore crosses a governance boundary. **Escalate** requests authority unavailable to the active workstream.

Local verification is automatic where a tool or predicate can check the action. Independent review is reserved for material evidence, claim freeze, release, or submission boundaries. Independence means a fresh context and no authorship of the audited artifact; merely repeating the same prompt does not remove shared blind spots.

5.2 Imperfect verification

Let a fault remain present at successive gates. Define M_i as the event that it survives gate i undetected.

Assumption 5.1 (Conditional miss bound). *For some $0 \leq \alpha < 1$, $\Pr(M_i \mid M_1 \cap \dots \cap M_{i-1}) \leq \alpha$ at every gate at which the fault is observable.*

Theorem 5.2 (Survival and detection bound). *Under the conditional miss bound,*

$$\Pr(M_1 \cap \dots \cap M_k) \leq \alpha^k.$$

If N is the number of gate intervals through which the fault persists before detection, then $\mathbb{E}[N] \leq (1 - \alpha)^{-1}$.

Proof. The chain rule and the conditional bounds give the product bound. The tail-sum formula gives $\mathbb{E}[N] = \sum_{k \geq 0} \Pr(N > k) \leq \sum_{k \geq 0} \alpha^k = (1 - \alpha)^{-1}$. $\square$

No independence assumption is needed beyond the stated conditional bound. The bound is useless when the fault is not represented in the evidence channel, a limitation formalized in [Definition 11.1](#).

5.3 Repair versus replan

A repair changes operational means while preserving the DIC’s semantics and authority. A replan changes at least one parent-level obligation, dependency, acceptance meaning, or frozen protocol. This distinction is auditable: a repair produces a new EPS under the same DIC hash; a replan changes a DIC or Plan revision and opens a checkpoint or escalation event.

6 Dependency Locality and Failure Propagation

6.1 Declared footprints

Model state as a key-value map $s : \mathcal{K} \rightarrow \mathcal{V}$. Each phase v declares read set $R_v \subseteq \mathcal{K}$ and write set $W_v \subseteq \mathcal{K}$. Edges include every possible data dependence: if $W_u \cap R_v \neq \emptyset$, then $u \rightarrow v$ unless an equivalent versioned external dependency is declared.

After a repair changes keys $\Delta \subseteq \mathcal{K}$, define the direct-reader seed

$$B(\Delta) = \{v : R_v \cap \Delta \neq \emptyset\}$$

and dynamic impact cone

$$\mathcal{I}(\Delta) = \text{Reach}_G(B(\Delta)),$$

including the seed.

Assumption 6.1 (Local deterministic-or-reverified semantics). *A phase's output and evidence validity depend only on its declared read values, declared predecessor outputs, explicit tool/version parameters, and fresh randomness recorded in evidence. There is no undeclared mutable shared state. Evidence is bound to input and artifact versions.*

Theorem 6.2 (Dependency-local recovery). *Under complete dependency edges and local deterministic-or-reverified semantics, after a repair that changes only Δ , retaining completed phase outputs and evidence for every $v \notin \mathcal{I}(\Delta)$ preserves their contract validity. Re-execution need be considered only inside $\mathcal{I}(\Delta)$.*

Proof. Take a topological order of nodes outside the cone. A node $v \notin \mathcal{I}(\Delta)$ does not directly read a changed key. Nor can any predecessor output on which it depends have changed: otherwise a changed predecessor would reach v , placing v in the cone. By induction, all declared inputs, tool/version parameters, and predecessor outputs of v are unchanged. Local semantics therefore preserve its output distribution under recorded randomness, or fresh reverification certifies it, and version-bound evidence remains valid. $\square$

Theorem 6.3 (Worst-case tightness). *Without additional semantic restrictions, every node in $\mathcal{I}(\Delta)$ can be necessary to invalidate: for any such node, there exists a set of phase functions consistent with the declared graph for which changing Δ changes that node's accepted output. Hence no strict subset of the impact cone is uniformly safe over the declared-footprint model.*

Proof. For a direct reader, choose its output to copy a changed bit. Along any path from that reader to a target descendant, choose each phase to copy the predecessor bit and let its acceptance predicate bind to the previous version. The change propagates to the target and invalidates its old evidence. Thus each reachable node is necessary in some admissible realization. $\square$

This result explains both the value and fragility of local repair. Complete dependency declarations can reduce rework from a full rerun to $\sum_{v \in \mathcal{I}(\Delta)} c_v$; one hidden edge can make that guarantee false. The structure parallels dependency graphs, incremental build systems, and self-adjusting computation (Ferrante et al., 1987; Mokhov et al., 2018; Acar, 2009).

6.2 Propagation bounds

A fault originating at node u can affect at most $|\text{Desc}(u)| + 1$ phases and in the worst case affects all of them. If the descendant subgraph has branching at most b and depth at most d , then

$$|\text{Desc}(u)| + 1 \leq \sum_{i=0}^d b^i,$$

with the usual $d + 1$ limit when $b = 1$. Verification gates do not alter this structural cone; they reduce how much of it executes before detection and which outputs remain trusted.

7 Context and Persistent State

7.1 Package as authority

Chat history is not a durable database. The package therefore stores a small authoritative index and immutable or versioned semantic records. A fresh workstream reads the approved goal, Plan, active Phase, complete multi-view DIC, EPS/prompt/action lineage, compact reports surface, evidence index, checkpoints, and escalations from the package. Generated narrative handoffs are useful views, not competing authorities.

7.2 Context projection

Let $\Pi_v(s)$ be the context projection supplied to the executor for phase v , and let $\pi_v(a \mid s)$ be the ideal phase action kernel.

Proposition 7.1 (Context-local execution). *Omitting state outside $\Pi_v(s)$ leaves phase behavior unchanged if*

$$\pi_v(a \mid s) = \pi_v(a \mid \Pi_v(s))$$

for all admissible s, a , all acceptance predicates and escalation conditions are measurable from the projection plus resolvable evidence references, and omitted state cannot mutate through a hidden channel. If two admissible states share a projection but require different action distributions or acceptance decisions, the projection is insufficient.

Proof. The sufficient direction follows by substitution in the action and verification kernels. For necessity, take states s, s' with $\Pi_v(s) = \Pi_v(s')$ but different required behavior; any projection-only executor must act identically on them and is wrong on at least one. $\square$

Externalized state reduces context load only when the projection is semantically sufficient. It is not made safe by storing more files whose relevance cannot be recovered. MemGPT and hierarchical memory systems motivate externalization and working-memory control (Packer et al., 2023; Wang et al., 2024); the DIC specifies what must be projected for one bounded phase.

7.3 DIC as a human-attention projection

The same projection problem applies to the researcher. Let $\Pi_H(s)$ denote the human-facing Phase state. Requiring the human to read every prompt, transcript, terminal log, failed branch, and test output makes supervisory burden grow with machine activity. The DIC instead exposes a compact current projection: Primary Aim, prioritized requests, durable topology, decisive evidence pointers, unresolved material questions, and authority boundaries. Detailed execution remains addressable by path when a decision requires it. The design principle is therefore *increasing machine complexity should not imply increasing human reconstruction complexity*. A compact DIC is not safe merely because it is short; it must remain sufficient for the material decisions delegated to the human.

8 Human Oversight Across Timescales

8.1 Human-attention intensity

Let $\mathcal{H}(\tau)$ be the human-attention events in run τ : approvals, material adjudications, context reconstruction, manual routing, evidence inspection, or other supervisory interventions. Let event j have duration t_j and a prespecified attention weight $w_j \geq 0$. Define

$$A_H(\tau) = \sum_{j \in \mathcal{H}(\tau)} w_j t_j.$$

When detailed timing is unavailable, event counts by intervention class are a lower-fidelity proxy. Let verified research progress be

$$P_V(\tau) = \sum_{v \in V} q_v z_v,$$

where $q_v > 0$ is a prespecified milestone weight and $z_v \in \{0, 1\}$ indicates that the milestone is accepted against current evidence. The *human-attention intensity*

$$\rho_H(\tau) = \frac{A_H(\tau)}{P_V(\tau) + \varepsilon}$$

measures human supervisory attention per unit of verified progress. Its reciprocal is an attention-efficiency score. Because progress units and weights are project-specific, ρ_H is meaningful primarily for matched tasks or within a frozen benchmark definition; it is not a universal scalar of research quality.

The workflow targets lower ρ_H by moving routine routing, state reconstruction, local repair, and evidence bookkeeping below the human boundary while preserving escalation for high-value judgement. This is an empirical objective, not a demonstrated result. A badly designed hierarchy can increase both A_H and error rates.

8.2 Governance complexity

Suppose a Plan has N ordinary completed phases, periodic checkpoint interval h , and M material escalations. Beyond initial Plan approval, the number of mandatory human governance events is at most

$$\left\lfloor \frac{N}{h} \right\rfloor + M$$

(up to a chosen final release review). This is a counting invariant of the policy, not a quality guarantee. Machine-local checks may be much more frequent. If each mandatory governance event has attention cost at most $\bar{a}$ and initial Plan review costs a_0 , the policy-level contribution is bounded by $a_0 + \bar{a}(\lfloor N/h \rfloor + M)$; optional deep investigations and difficult escalations may exceed this bound and should be measured separately.

8.3 Checkpoint interval under imperfect verification

Consider a simplified stationary model. Each checkpoint costs c_v ; a false rejection occurs with probability β and costs c_f ; faults arrive at rate λ per phase; re-executing one contaminated phase costs c_r ; a persisting observable fault is missed at a gate with conditional probability $\alpha < 1$. With interval h , a fault appears uniformly within its first block. Its expected contaminated span before detection is

$$\frac{h}{2} + h \frac{\alpha}{1 - \alpha} = h \frac{1 + \alpha}{2(1 - \alpha)}.$$

Ignoring overlapping faults and edge effects, expected overhead over n phases is

$$U(h) = \frac{n(c_v + \beta c_f)}{h} + \lambda n c_r h \frac{1 + \alpha}{2(1 - \alpha)}.$$

Proposition 8.1 (Cost-optimal interval in the stationary model). *The continuous minimizer is*

$$h^* = \sqrt{\frac{2(c_v + \beta c_f)(1 - \alpha)}{\lambda c_r (1 + \alpha)}}.$$

The discrete policy uses a feasible integer near h^ .*

Proof. U is strictly convex for $h > 0$. Setting its derivative to zero and solving gives the expression. $\square$

The formula recovers the qualitative checkpoint/restart trade-off (Young, 1974; Daly, 2006): expensive review favors longer intervals; frequent or costly faults favor shorter intervals; a weak verifier changes both detection delay and repair overhead. The runtime default of seven is a configurable operating prior, not an estimate of h^* for every project.

9 Main, Theory, Experiment, and Coding-Agent Orchestration

Specialization is semantic, not ceremonial. Main is the persistent user-facing chat that preserves the Primary Aim, integrates specialist evidence, owns the Plan and one whole-Phase DIC, and decides material scientific or authority changes. Theory is an optional specialist reasoning workstream for formal reasoning, counterexamples, and novelty. Experiment is an optional specialist reasoning workstream for estimands, frozen protocols, controls, and interpretation. Research is an advisory evidence layer that returns to Main before becoming binding.

The coding-agent Orchestrator is operationally distinct. It receives the approved DIC, resolves the actual execution model/surface, compiles model-tuned EPS prompts, chooses direct execution versus bounded Workers/subagents, manages safe parallelism and write ownership, integrates returns, performs contract-preserving local repair, and dispatches verification. Routine execution detail should remain below Main unless it changes scientific meaning or decisive evidence. Worker completion is not Orchestrator integration, and integration is not Verifier acceptance.

This separation also protects salience. Theory should not accumulate available lemmas that do not strengthen or falsify the Primary Aim; Experiment should not expand benchmark matrices merely to appear comprehensive; Orchestrator should not optimize for safe wording, passing one convenient test, or maximal activity when those goals weaken the DIC. Negative evidence is preserved and used to repair or replan rather than converted into defensive narrative.

The separation reduces a common failure in which one context invents a claim, designs a favorable test, interprets the result, and approves publication. It does not guarantee epistemic independence. Material theory and evidence should receive a fresh hostile review at a release boundary.

10 Reference Implementation

10.1 Authority and files

The v5.6.37 reference runtime uses the Python standard library for its canonical state machine. `.ai-workflow/STATE.json` records schema version, active Plan, phase summaries, DIC identity/hash, EPS attempts, Actions, verification outcomes, and evidence pointers. Human-readable DIC views are materialized under `phases/<phase>/DIC/`; EPS and Action lineage remains indexed in canonical state and may point to materialized prompts or evidence files. Exactly one DIC is permitted per Phase, and its three Core views are required. The runtime checks DIC graph acyclicity, EPS–DIC identity/hash consistency, Action lineage, verification outcomes, and package recovery. Model-executed EPS nodes cannot record Actions until the selected model/surface and prompt-guidance profile are recorded and the prompt is marked tuned. These are implementation invariants, not semantic guarantees about the world.

10.2 State transitions

The principal transitions are:

Transition	Preconditions	Result
create Plan	initialized package	draft Plan
add Phase	draft Plan	dependency-checked Phase
approve Plan	at least one Phase	approved authority boundary
start Phase	approved, dependencies complete	active Phase/DIC
generate EPS	DIC/PLAN exists	disposable prompt/run instantiation
complete Phase	active, acceptance satisfied	evidence + next phase/checkpoint
verify repair	same DIC remains valid	regenerate EPS
verify replan	semantic decomposition changes	governance event
escalate	material trigger	autonomous progression stops
recover	package exists	lineage/hash/dependency audit

10.3 Compatibility

Legacy v5.6.31 Goal/context/handoff files are indexed rather than deleted. **Orchestrator** aliases Main; **Researcher** aliases Theory by default or Experiment for empirical work. A legacy single prompt can be migrated as a one-Phase Plan. The compatibility layer is deliberately subordinate to the canonical state to prevent multiple status files from claiming authority.

10.4 Conformance coverage

The test suite covers one Plan authorizing several phases, autonomous ordinary progression, periodic checkpoint triggering, immediate material escalation, reconstructable lineage, Main/Theory/Experiment ownership, package-only recovery, and legacy compatibility, plus dependency blocking and verification outcomes. These are engineering tests. They do not measure language-agent task success.

11 Formal Properties and Limits

11.1 What is guaranteed by construction

Identifier lineage, role normalization, checkpoint counters, escalation categories, atomic state replacement, and DIC hash checks are implementation invariants when callers use the canonical runtime. They are not semantic guarantees about the world.

11.2 Observability limit

Let $O : \mathcal{S} \rightarrow \mathcal{O}$ be the evidence observation available to a verifier and $G : \mathcal{S} \rightarrow \{0, 1\}$ the relevant goal predicate.

Proposition 11.1 (Evidence-aliasing impossibility). *If there exist s_+, s_- such that $O(s_+) = O(s_-)$ but $G(s_+) = 1$ and $G(s_-) = 0$, then no verifier $V : \mathcal{O} \rightarrow \{\text{accept}, \text{reject}\}$ can have both zero false acceptance and zero false rejection on $\{s_+, s_-\}$.*

Proof. V receives the same input on both states and must return the same decision. Accepting falsely accepts s_- ; rejecting falsely rejects s_+ . $\square$

No amount of hierarchical decomposition overcomes evidence that fails to distinguish success from failure. The remedy is a richer environment-grounded observation, a weaker claim, or explicit residual uncertainty—not a more confident verifier prompt.

11.3 Open-world and adversarial limits

The theory assumes declared dependencies, stable identifiers, and evidence tied to the asserted property. Open-world side effects, prompt injection, mutable external services, and adversarial evidence can violate those assumptions. ToolEmu and AgentDojo illustrate why sandbox and security evaluation are separate requirements (Ruan et al., 2024; Debenedetti et al., 2024). The runtime is a control layer, not a security boundary.

11.4 Novelty limit

The formal ingredients have established analogues in program logics, runtime verification, build systems, and checkpointing (Hoare, 1969; Leucker and Schallhart, 2009; Mokhov et al., 2018; Daly, 2006). The report therefore claims neither theorem novelty in isolation nor implementation equivalence to a learned hierarchical world model. LeCun’s world-model programme and joint-embedding predictive models are conceptual motivation for abstraction across timescales, not descriptions of this symbolic package protocol (LeCun, 2022; Bardes et al., 2024).

12 Empirical Hypotheses and Evaluation Design

12.1 Central hypotheses

The primary empirical hypothesis is that the relative benefit of hierarchy increases with effective task horizon after matching model, task information, and inference budget. The mechanism hypothesis is that verification-triggered local repair reduces downstream propagation and rework beyond hierarchy alone and beyond spending the same calls on generic critique.

12.2 Conditions

The frozen candidate design contains four main conditions: C0 direct/reactive; C1 flat plan-and-execute; C2 hierarchical execution; and C3 hierarchy plus evidence-gated repair/replanning. C1+V spends C3’s verifier budget on generic critique without dependency-local repair; C2-extra spends it on additional generation. These controls distinguish structure from extra tokens, calls, stronger prompts, or verifier-as-additional-inference.

12.3 Task factors and faults

Controlled factors are dependency depth, branching, delayed dependency distance, interacting constraints, and injected intermediate faults. Task families should have objective executable graders and pre-generated instances. Faults include stale schemas, omitted constraints, dependency-version swaps, corrupted statistics, unsupported citation pointers, and failing code paths. The injector hides fault existence and location from the agent.

12.4 Outcomes and analysis

Primary outcomes are end-to-end success, hard-constraint satisfaction, unsupported completion, failure-propagation size, recovery locality, detection latency, and rework. Resource measures include calls, tokens, wall time, verifier calls, and tool use. Human-supervision outcomes include attention-event counts by class, measured or logged intervention duration where feasible, context-reconstruction episodes, manual routing events, and human-attention intensity ρ_H under prespecified milestone weights. The primary statistical analysis is a condition-by-effective-horizon interaction in a mixed-effects model, with paired instance contrasts and full interval estimates. A positive submission-level claim requires frozen prompts, matched budgets, at least two task families, objective graders, and completion of all primary conditions.

12.5 Current evidence boundary

The current reconstruction includes conformance tests and a deterministic dependency-locality fixture. They validate implementation paths and terminology only. No large confirmatory comparative study has been run, so the package does not claim empirical superiority.

13 Related Work

Planning and search. ReAct, Tree of Thoughts, Graph of Thoughts, ReWOO, ADaPT, and LATS develop reasoning/action interleaving, explicit search, or adaptive decomposition (Yao et al., 2023b,a; Besta et al., 2024; Xu et al., 2023; Prasad et al., 2024; Zhou et al., 2024a). AI-Native Workflow operates at a longer-lived authority and artifact layer; it can host such methods inside EPS.

Compound agents. CAMEL, AutoGen, AgentVerse, and MetaGPT establish role-based multi-agent coordination (Li et al., 2023; Wu et al., 2023; Chen et al., 2024; Hong et al., 2024). The present focus is durable refinement, evidence, invalidation, and governance rather than the novelty of manager/worker roles.

Memory and context. MemGPT and HiAgent address external or hierarchical memory (Packer et al., 2023; Wang et al., 2024). The context-locality proposition states the sufficiency condition needed for safe omission; it does not propose a new neural memory model.

Benchmarks. AgentBench, WebArena, GAIA, SWE-bench, OSWorld, and τ -bench expose stateful, tool-mediated, and long-horizon failures (Liu et al., 2024; Zhou et al., 2024b; Mialon et al., 2024; Jimenez et al., 2024; Xie et al., 2024; Yao et al., 2024). The proposed factorial generator complements them by manipulating dependency structure and fault location directly.

Verification and oversight. Reflexion, process supervision, debate, recursive oversight, and weak-to-strong supervision motivate feedback and scalable checking (Shinn et al., 2023; Lightman et al., 2023; Irving et al., 2018; Leike et al., 2018; Burns et al., 2024). The four-outcome operator adds explicit repair/replan/escalation semantics, while the observability proposition prevents treating verifier confidence as ground truth.

Harness optimization. DSPy and automated design of agentic systems optimize prompts or agent graphs (Khattab et al., 2024; Hu et al., 2025). AI-Native Workflow specifies how a chosen harness is authorized, executed, audited, and locally recovered. Direct comparative evidence is future work.

14 Limitations

The architecture increases explicit state and may add overhead on short tasks. It may also shift rather than reduce human burden if DICs are verbose, ambiguous, or require frequent reconstruction. DIC quality is itself model- and human-dependent. Read/write footprints can be incomplete; hidden dependencies invalidate local recovery. Evidence can be stale, adversarial, or weakly connected to the claim. Verifier errors may be correlated, violating convenient rate models. The checkpoint optimum assumes stationarity and ignores overlapping faults, queueing, and heterogeneous costs. Human-attention intensity depends on project-specific progress weights and imperfect proxies for cognitive effort, so it should be compared only under frozen task definitions rather than interpreted as a universal productivity measure. Role specialization does not create epistemic independence. The package has no confirmatory evidence yet that the

proposed hierarchy improves end-to-end agent performance. Workshop templates and deadlines are external mutable constraints and must be rechecked immediately before submission.

15 Conclusion

AI-Native Workflow 5.6.37 formalizes a pattern researchers already assemble informally from chat, research, and coding-agent tools. A human and Main preserve the programme’s Primary Aim and material authority; one DIC per Phase keeps the scientific contract and execution topology intelligible; the coding-agent Orchestrator realizes that topology through model-tuned EPS attempts and bounded subagents; evidence-gated verification chooses accept, repair, replan, or escalate; and declared dependencies make local recovery auditable. The formal analysis identifies what composition and locality require, quantifies imperfect-verification/checkpoint trade-offs, and defines human-attention intensity as an explicit evaluation target. The next scientific step is controlled measurement: hold model and task information fixed, vary effective horizon and faults, and determine whether the integrated mechanism improves trustworthy progress while reducing the human attention spent on routine coordination.

A Claim classification summary

ID	Classification	Status
ARCH-01/02	invariant by construction	implemented and test-covered
DEF-REF-01	definition	retained
THM-COMP-01	theorem under assumptions	proved; not novelty claim
THM-LOCAL-01/02	theorems under assumptions	proved; locality and tightness
THM-VER-01	theorem under assumptions	proved; conditional miss bound
PROP-CHK-01	proposition under assumptions	proved; stationary model
PROP-CTX-01	proposition under assumptions	proved; conditional sufficiency
IMP-OBS-01	impossibility proposition	proved
HYP-HORIZON-01	empirical hypothesis	planned, not tested
HYP-REPAIR-01	empirical hypothesis	planned, not tested

B Required experiment record

Each run record should include package hash, condition, model identifier, decoding parameters, tool versions, task seed, dependency graph, hidden fault metadata, transcript pointer, artifact hashes, grader version, raw outcomes, cost measures, and termination reason. Aggregate manuscript tables must be generated from those records.

C Ordinary execution brief

A modern prompt needs the desired outcome, package/domain authority, hard constraints, evidence and success criteria, autonomy/escalation boundary, and deliverable. The model chooses the route and records artifacts and concise rationale; it is not asked to reveal private chain-of-thought.

References

- Umut A. Acar. *Self-Adjusting Computation*. Morgan & Claypool, 2009.
- Adrien Bardes, Quentin Garrido, Jean Ponce, Michael Rabbat, Yann LeCun, Mahmoud Assran, and Nicolas Ballas. V-jepa: Latent video prediction for visual representation learning. In *International Conference on Machine Learning*, 2024.
- Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, and Torsten Hoefer. Graph of thoughts: Solving elaborate problems with large language models. In *AAAI Conference on Artificial Intelligence*, 2024.
- Collin Burns, Haotian Ye, Dan Klein, and Jacob Steinhardt. Weak-to-strong generalization: Eliciting strong capabilities with weak supervision. In *International Conference on Machine Learning*, 2024.
- Weize Chen, Yusheng Su, Jingwei Zuo, Cheng Yang, Chenfei Yuan, Chi-Min Chan, Heyang Yu, Yaxi Lu, Yi-Hsin Hung, Chen Qian, et al. Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In *International Conference on Learning Representations*, 2024.
- John T. Daly. A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3):303–312, 2006.
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents. In *Advances in Neural Information Processing Systems*, 2024.
- Jeanne Ferrante, Karl J. Ottenstein, and Joe D. Warren. Program dependence graphs and their use in optimization. In *ACM Transactions on Programming Languages and Systems*, 1987.
- C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.
- Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, et al. Metagpt: Meta programming for multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. In *International Conference on Learning Representations*, 2025.
- Geoffrey Irving, Paul Christiano, and Dario Amodei. Ai safety via debate. *arXiv preprint arXiv:1805.00899*, 2018.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, 2024.

- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, et al. Dspy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*, 2024.
- Yann LeCun. A path towards autonomous machine intelligence. *OpenReview preprint*, 2022.
- Jan Leike, David Krueger, Tom Everitt, Miljan Martic, Vishal Maini, and Shane Legg. Scalable agent alignment via reward modeling: A research direction. *arXiv preprint arXiv:1811.07871*, 2018.
- Martin Leucker and Christian Schallhart. A brief account of runtime verification. *Journal of Logic and Algebraic Programming*, 78(5):293–303, 2009.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. Camel: Communicative agents for mind exploration of large scale language model society. In *Advances in Neural Information Processing Systems*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. *arXiv preprint arXiv:2305.20050*, 2023.
- Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. Agentbench: Evaluating llms as agents. In *International Conference on Learning Representations*, 2024.
- Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. Gaia: A benchmark for general ai assistants. In *International Conference on Learning Representations*, 2024.
- Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. In *Proceedings of the ACM on Programming Languages*, 2018.
- Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, and Tushar Khot. Adapt: As-needed decomposition and planning with language models. In *Findings of the Association for Computational Linguistics: NAACL*, 2024.
- Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Toolemu: Identifying the risks of lm agents with an lm-emulated sandbox. In *International Conference on Learning Representations*, 2024.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- Zhenhailong Wang, Shaoguang Liu, Yixin Chen, Zhaoran Zheng, Mingyuan Wang, Kenji Kawaguchi, and Huaxiu Yao. Hiagent: Hierarchical working memory management for solving long-horizon agent tasks with large language models. *arXiv preprint*, 2024.
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

- Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Tenyue Hua, Zhoujun Cheng, Dong-Suk Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems*, 2024.
- Binfeng Xu, Zhendong Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. Rewoo: Decoupling reasoning from observations for efficient augmented language models. *arXiv preprint arXiv:2305.18323*, 2023.
- Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023a.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023b.
- Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.
- John W. Young. A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9):530–531, 1974.
- Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *International Conference on Machine Learning*, 2024a.
- Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. Webarena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024b.